

Clase 15. Bootcamp AI Engineer

RAG Evals + DSPy

Ramsés Camas · Bootcamp AI
Engineer



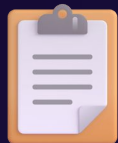


Tabla de contenido

Clase 15

/01

Métricas RAG

Faithfulness, P/R, relevancy.

/02

Eval. agentes

Tool calls + trayectoria.

/03

DSPy

Compile prompts vs tuneat.

/04

Pipeline final

Evals → compile → re-evals.

Recap clase 14

Lo que ya tenemos en el quality gate.

Pero solo contestamos:
¿cambió algo? — no: ¿qué tan bueno es?



- **Contratos:** el output tiene la forma correcta (Pydantic + pytest).
- **Golden tests:** el output se parece a una referencia.
- **Replay de trazas:** el comportamiento histórico se mantiene.
- **Regresión:** un cambio no degradó nada.
- **Datasets + runner:** curados, sintéticos, ejecución local con Makefile.



**“Cambió
≠ está bien.”**

— Forma vs sustancia

Taxonomía de evals

	Pregunta	Herramienta	Cubierto	Estado
Contratos	¿Forma correcta?	Pydantic + pytest	Clase 14	✓
Golden	¿Output similar a ref?	embeddings + keywords	Clase 14	✓
RAG metrics	¿Retrieval grounded?	RAGAS / LLM-as-judge	Hoy	→
Agent evals	¿Tools y plan correctos?	matchers + trajectory	Hoy	→
Optimización	¿Cómo mejoro guiado por métricas? — DSPy compile (el cierre de hoy).			

El "no inventes"

El detector #1 de alucinación en RAG.

Mide: proporción de claims en la respuesta respaldados por el contexto recuperado.
Rango 0.0 → 1.0.



- **Por qué importa:** una respuesta puede ser fluida y completamente fabricada.
- **Sin faithfulness:** tu RAG es un LLM con texto ambiental, no grounded.
- **1.0:** todos los claims respaldados por el contexto.
- **0.0:** todo inventado, ningún claim verificable.
- **Modo de fallo típico:** el modelo rellena huecos con conocimiento del pre-training.

Cómo se mide

Pipeline en 3 pasos. Es LLM-as-judge: usamos un LLM para juzgar el output de otro LLM.

Trampa: si el judge es el mismo modelo, hay sesgo. Usa un modelo distinto o más fuerte como juez.



- **Paso 1 — Descomponer:** un LLM trocea la respuesta en claims atómicos.
- **Paso 2 — Verificar:** otro LLM responde sí/no por cada claim contra el contexto.
- **Paso 3 — Promediar:** $\text{faithfulness} = \frac{\text{claims_respaldados}}{\text{total_claims}}$.
- **Sesgo de juez:** mismo modelo = se autoaprueba. Usa un juez distinto.
- **Costo:** $N+1$ llamadas por respuesta evaluada. Cachea claims.

```
from groq import Groq
import json

client = Groq()
JUDGE_MODEL = "llama-3.3-70b-versatile"

def extract_claims(answer: str) -> list[str]:
    resp = client.chat.completions.create(
        model=JUDGE_MODEL,
        messages=[{"role": "user", "content":
            f"Descompon en claims atomicos (JSON list):\n{answer}"}],
        response_format={"type": "json_object"},
    )
    return json.loads(resp.choices[0].message.content)["claims"]

def verify_claim(claim: str, context: str) -> bool:
    resp = client.chat.completions.create(
        model=JUDGE_MODEL,
        messages=[{"role": "user", "content":
            f"El contexto respalda el claim? Responde SI o NO.\n"
            f"Contexto: {context}\nClaim: {claim}"}],
    )
    return "SI" in resp.choices[0].message.content.upper()

def faithfulness(answer: str, context: str) -> float:
    claims = extract_claims(answer)
    if not claims: return 0.0
    return sum(verify_claim(c, context) for c in claims) / len(claims)
```



Context Precision y Context Recall

Context Precision: de los chunks recuperados, ¿qué proporción son realmente relevantes? Penaliza retrievers que traen ruido. Útil cuando el generador es débil o cuando el contexto se cobra por token (latencia, dinero).

Context Recall: de la información necesaria para responder, ¿qué proporción está en el contexto recuperado? Penaliza retrievers que dejan huecos. Útil en dominios donde omitir es peor que sobrar (legal, médico). Tradeoff clásico: subir top_k mejora recall pero baja precision.

Cuándo te importa cada una



Precision pesa cuando

El generador es débil. El contexto cuesta tokens. Estás en agentic RAG con varios pasos. DocOps Agent: precision pesa más, porque el agente decide acciones.



Recall pesa cuando

Las preguntas requieren juntar información dispersa. Dominios donde omitir es peor que sobrar (legal, médico). Cuando evalúas si vale re-ranear.

```
def context_precision(retrieved: list[str], question: str) -> float:
    """Que proporcion de chunks son relevantes a la pregunta?"""
    if not retrieved: return 0.0
    relevances = []
    for chunk in retrieved:
        resp = client.chat.completions.create(
            model=JUDGE_MODEL,
            messages=[{"role": "user", "content":
                f"Este chunk es relevante a la pregunta? SI/NO.\n"
                f"Pregunta: {question}\nChunk: {chunk}"}],
        )
        relevances.append("SI" in resp.choices[0].message.content.upper())
    return sum(relevances) / len(relevances)

def context_recall(retrieved: list[str], ground_truth: str) -> float:
    """Que proporcion de claims del ground_truth estan en el contexto?"""
    claims = extract_claims(ground_truth)
    if not claims: return 0.0
    context = "\n".join(retrieved)
    return sum(verify_claim(c, context) for c in claims) / len(claims)
```



Va al punto

¿La respuesta realmente contesta lo que se preguntó, o se va por las ramas?

Detecta el modo de fallo "respuesta correcta a la pregunta equivocada", que faithfulness no captura.



- **Truco de medición:** generar N preguntas a partir de la respuesta.
- **Comparar:** embeddings de esas preguntas vs la pregunta original.
- **Alta similitud:** la respuesta apunta a la pregunta original.
- **Baja similitud:** la respuesta habla de otra cosa.
- **Bonus:** no necesita ground truth, solo pregunta y respuesta.

Pregunta al RAG → RAG Responde →
Evaluar Respuesta → Generar N Preguntas
→ Comparar Preguntas a la pregunta original

```
from sentence_transformers import SentenceTransformer
import numpy as np

embedder = SentenceTransformer(
    "paraphrase-multilingual-MiniLM-L12-v2"
)

def answer_relevancy(answer: str, question: str, n: int = 5) -> float:
    resp = client.chat.completions.create(
        model=JUDGE_MODEL,
        messages=[{"role": "user", "content":
            f"Genera {n} preguntas que esta respuesta contestaria "
            f"(JSON list):\n{answer}"}],
        response_format={"type": "json_object"},
    )
    generated = json.loads(resp.choices[0].message.content)["questions"]

    q_emb = embedder.encode(question)
    g_embs = embedder.encode(generated)
    sims = [
        np.dot(q_emb, g) / (np.linalg.norm(q_emb) * np.linalg.norm(g))
        for g in g_embs
    ]
    return float(np.mean(sims))

# Modelo de embeddings local: cero costo, multilingue.
```



RAGAS: ¿lo construyo o lo uso?



Usar RAGAS

Métricas ya implementadas y probadas. Integra con Groq vía langchain-groq. Reportes y dashboards out-of-the-box.

Bueno para producción cuando ya entiendes qué hace.

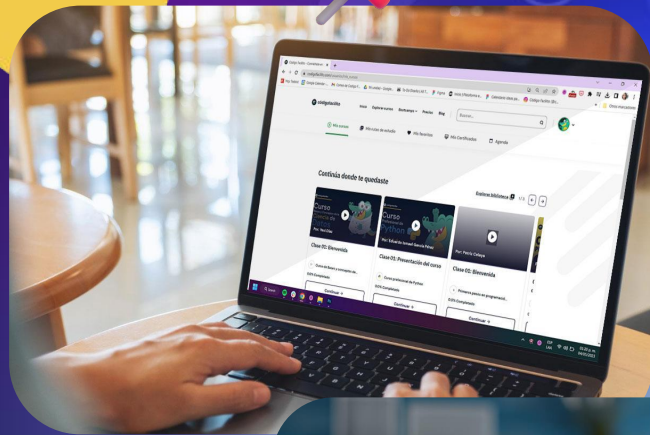


Construirlo

Entiendes cada métrica desde adentro. Cero dependencias mágicas. Adaptable a métricas de tu dominio. Bootcamp: construir en clase, usar RAGAS después.

/02 Agent Evals

[Más allá del output]



Tool calls

Hasta aquí evaluamos outputs. Un agente tiene proceso. Necesitamos evals de trayectoria.

Es el equivalente a un test unitario, pero del plan del agente.



- **Tool match:** ¿llamó la herramienta correcta (search_docs vs create_ticket)?
- **Argument match:** ¿los argumentos coinciden (exact, fuzzy, semantic)?
- **Order match:** ¿la secuencia de tools es la esperada?
- **DocOps Agent:** validar query_rewrite → retrieve → synthesize.
- **Granularidad:** puedes medir cada componente o el plan completo.

```
from dataclasses import dataclass

@dataclass
class ExpectedCall:
    tool: str
    args: dict # exact match en keys, fuzzy en values

def tool_call_accuracy(
    actual: list[dict],
    expected: list[ExpectedCall],
) -> dict:
    if len(actual) != len(expected):
        return {"tool_match": 0.0, "arg_match": 0.0, "order_match": 0.0}

    tool_hits, arg_hits = 0, 0
    for a, e in zip(actual, expected):
        if a["tool"] == e.tool:
            tool_hits += 1
        if set(a["args"].keys()) == set(e.args.keys()):
            arg_hits += 1
    n = len(expected)
    return {
        "tool_match": tool_hits / n,
        "arg_match": arg_hits / n,
        "order_match": 1.0 if tool_hits == n else 0.0,
    }
```



Pasos del agente

Más allá del match exacto, medimos eficiencia y coherencia de la trayectoria.

Esto convierte a tu agente en algo debuggable, no en una caja negra estocástica.



- **Step count:** ¿cuántos pasos tomó vs el óptimo conocido?
- **Redundant calls:** ¿repitió tools innecesariamente?
- **Recovery rate:** cuando una tool falló, ¿se recuperó?
- **Plan-execution gap:** ¿lo planeado coincide con lo ejecutado?
- **Sin LangSmith ni Phoenix:** con tus propias trazas en JSON local.

```
def evaluate_trajectory(trace: list[dict], optimal_steps: int) -> dict:
    tool_calls = [s for s in trace if s["type"] == "tool_call"]
    tools_used = [c["tool"] for c in tool_calls]

    redundancy = len(tools_used) - len(set(tools_used))
    failures = [c for c in tool_calls if c.get("error")]

    recoveries = 0
    for c in failures:
        idx = tool_calls.index(c)
        if idx + 1 < len(tool_calls) and not tool_calls[idx + 1].get("error"):
            recoveries += 1

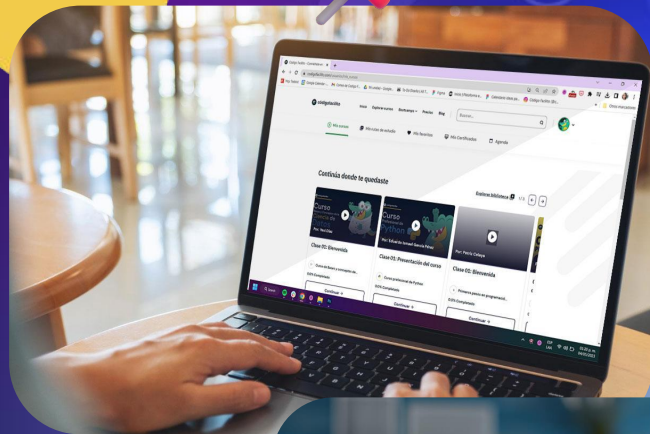
    return {
        "step_efficiency": optimal_steps / max(len(tool_calls), 1),
        "redundancy": redundancy,
        "recovery_rate": recoveries / len(failures) if failures else 1.0,
    }
```

Sin servicios externos. JSON traces que tu mismo escribes en cada nodo.



/03 DSPy

[Programar, no prompt-tunear]



Prompt manual

Mejorar tu sistema = abrir el archivo de prompts y editar a mano. Esto tiene tres problemas estructurales.

¿Y si el prompt fuera código compilado contra una métrica? Ese es DSPy.



- **No escala:** 20 prompts × 5 modelos × 3 tareas = inmanejable.
- **No es reproducible:** el "buen prompt" depende del humor del developer.
- **No usa la señal:** tienes métricas, pero no las usas para guiar mejoras.
- **Versionado pobre:** los prompts viven en strings, no en CI.
- **Acoplamiento:** cambiar el modelo te obliga a re-tunear todo.



**“Programming,
not prompting.”**

— DSPy · Stanford NLP

```
import dspy

# Una Signature declara la INTERFAZ de un componente:
# inputs y outputs con tipos y descripciones.
# No escribes el prompt. Lo declaras. DSPy lo construye y optimiza.

class AnswerWithContext(dspy.Signature):
    """Responde la pregunta usando solo el contexto provisto."""

    context: str = dspy.InputField(desc="Documentos recuperados")
    question: str = dspy.InputField()
    answer: str = dspy.OutputField(
        desc="Respuesta concisa basada en el contexto"
    )

class RewriteQuery(dspy.Signature):
    """Reformula la query del usuario para mejorar el retrieval."""

    user_query: str = dspy.InputField()
    rewritten: str = dspy.OutputField(
        desc="Query optimizada para BM25 + embeddings"
    )
```



```
# Un Module envuelve una Signature con una estrategia de inferencia.

# Predict: llamada directa
qa_simple = dspy.Predict(AnswerWithContext)

# ChainOfThought: agrega un paso de razonamiento previo
qa_cot = dspy.ChainOfThought(AnswerWithContext)

# ReAct: patron reasoning+acting con tools
qa_agent = dspy.ReAct(AnswerWithContext, tools=[search_docs, create_ticket])

# ProgramOfThought: genera codigo y lo ejecuta
calc = dspy.ProgramOfThought("question -> answer")

# Uso: como cualquier callable
result = qa_cot(context=docs, question="Cual es la politica de refunds?")
print(result.answer)
print(result.rationale) # ChainOfThought expone el razonamiento
```



Compile

Un Optimizer toma tu programa, tu métrica y tu dataset, y produce un programa mejorado.

Output: el mismo programa con prompts optimizados, demos seleccionados, y a veces hasta instrucciones reescritas.



- **BootstrapFewShot:** genera ejemplos few-shot a partir del trainset.
- **+ RandomSearch:** prueba combinaciones, se queda con la mejor.
- **MIPROv2:** optimiza instrucciones y demos. SOTA en la mayoría de tareas.
- **COPRO:** optimiza solo instrucciones (más barato, menos potente).
- **Recomendación:** BootstrapFewShot para iterar, MIPROv2 para producción.

```
import os
import dspy

# Configurar Groq como LLM principal
lm = dspy.LM(
    "groq/llama-3.3-70b-versatile",
    api_key=os.environ["GROQ_API_KEY"],
    max_tokens=1024,
)
dspy.configure(lm=lm)

# Verificacion rapida
test = dspy.Predict("question -> answer")
print(test(question="Que es DSPy en una linea?").answer)

# Para gpt-oss-120b:
# lm = dspy.LM("groq/openai/gpt-oss-120b", api_key=...)

# Cero configuracion extra: DSPy habla OpenAI-compatible,
# y Groq lo expone igual.
```



```
# Para que DSPy optimice, necesita una metrica
# que reciba (example, prediction) y devuelva un score.

def docops_metric(example, prediction, trace=None) -> float:
    f = faithfulness(prediction.answer, example.context)
    r = answer_relevancy(prediction.answer, example.question)
    return 0.6 * f + 0.4 * r# combinada, ponderada

# Esta es exactamente la metrica que ya construimos en las slides 8 y 13.
# Reutilizacion pura: tu trabajo de evals se vuelve la funcion objetivo.

# Tip: si trace esta presente, DSPy te pasa la trayectoria interna.
# Util para penalizar tambien el costo (tokens) o latencia:

def docops_metric_with_cost(example, prediction, trace=None) -> float:
    quality = (0.6 * faithfulness(prediction.answer, example.context)
               + 0.4 * answer_relevancy(prediction.answer, example.question))
    cost_penalty = 0.001 * len(trace) if trace else 0
    return quality - cost_penalty
```




```
from dsp.teleprompt import MIPROv2

# Dataset desde la clase 14
trainset = load_dataset("evals/datasets/docops_train.jsonl")
valset = load_dataset("evals/datasets/docops_val.jsonl")

# Programa base (sin optimizar)
qa_baseline = dsp.ChainOfThought(AnswerWithContext)

# Optimizador
optimizer = MIPROv2(metric=docops_metric, auto="light")

# Compilacion
qa_optimized = optimizer.compile(
    qa_baseline,
    trainset=trainset,
    valset=valset,
)

# Guardar el programa optimizado (versionable en git)
qa_optimized.save("dsp_lab/qa_optimized.json")

# Un solo compile() reemplaza horas de prompt-tweaking manual.
```



Caso + traps

Componente: el query rewriter del DocOps Agent. Chico, métrica clara (context_precision), fácil de medir aislado.

Pipeline: baseline → métrica → compile MIPROv2 sobre 50 queries → comparar precision@5. Resultado típico: +10-25%.



- **Anti-patrón 1 DSPy:** optimizar sin val set → overfitting, métricas infladas.
- **Anti-patrón 2 DSPy:** métrica mal definida → DSPy optimiza lo que mides.
- **Anti-patrón 3 Eval:** judge = modelo del sistema → sesgo sistemático.
- **Anti-patrón 4 DSPy:** compilar sobre 5 ejemplos → sin señal. Mínimo 30-50.
- **Anti-patrón 5 DSPy:** no versionar el programa optimizado → no reproducible.

Resumen

Tu arsenal: faithfulness, P/R, relevancy, tool call y trajectory, DSPy compile.

Próxima (16): observabilidad, guardrails, deploy. Cierre del bootcamp.



- [evals/metrics.py](#): métricas RAG (faithfulness, P/R, relevancy).
- [evals/agent_metrics.py](#): tool call accuracy y trajectory.
- [dspy_lab/signatures.py](#): signatures del query rewriter.
- [dspy_lab/optimize.py](#): pipeline de compile con MIPROv2.
- [evals/reports/](#): reporte antes vs después del compile.